Implementation Note

1. Language Identification (LID)

Design Choice: Use of frame-level MFCC features with a BiLSTM model trained on dummy labels.

Instead of performing utterance-level classification, the system uses **frame-level LID**, allowing finer granularity in detecting language switches within a single sentence. This is important for code-switched speech where languages alternate frequently.

A non-obvious constraint was the lack of annotated Hinglish datasets. To address this, **dummy labels were used to validate the pipeline architecture rather than model accuracy**. The focus was on ensuring correct tensor flow, sequence modeling, and integration with downstream modules.

2. Speech-to-Text (STT)

Design Choice: Vocabulary-constrained decoding on top of Whisper output.

Rather than modifying the Whisper model itself, a lightweight **post-processing constraint mechanism** was applied. Words belonging to a predefined technical vocabulary were selectively emphasized.

This avoids expensive model fine-tuning while still improving recognition of domain-specific terms. The trade-off prioritizes **practical deployment over theoretical optimality**, which is important in low-resource scenarios.

3. IPA Conversion

Design Choice: Use of G2P (grapheme-to-phoneme) instead of rule-based phonetic mapping.

Instead of designing custom phonetic rules for Hinglish, the system uses g2p_en to convert text into phoneme sequences. This abstracts away linguistic complexity and ensures consistency.

The non-obvious insight here is that **approximate phonetic consistency is sufficient for downstream alignment**, even if the mapping is not linguistically perfect.

4. Translation

Design Choice: Dictionary-based translation instead of neural machine translation.

Due to the absence of a parallel dataset for Hinglish → low-resource language, a **controlled dictionary mapping** was used. While simplistic, this ensures deterministic and interpretable outputs.

This design prioritizes **pipeline completeness and explainability** over translation quality, which is acceptable in a prototype system.

5. Voice Synthesis (TTS)

Design Choice: Simulated waveform generation instead of full TTS model.

Modern TTS systems require large dependencies (PyTorch + pretrained models) which caused compatibility issues in the environment. Instead, a **placeholder waveform** was generated to preserve pipeline continuity.

This ensures that **downstream modules (DTW, prosody analysis)** can still be executed without breaking the system.

6. Prosody Alignment (DTW)

Design Choice: Use of pitch (F0) contours instead of raw waveform alignment.

Instead of aligning raw audio signals, **fundamental frequency (F0)** was extracted and aligned using DTW. This reduces dimensionality and focuses on prosodic features.

The key insight is that **prosody (intonation and rhythm) is better represented in F0 space**, making DTW more meaningful and computationally efficient.

7. Adversarial Robustness

Design Choice: Noise-based FGSM approximation instead of true gradient-based attack.

Applying true FGSM on sequence models caused instability due to gradient shape mismatches. Therefore, **Gaussian noise perturbation was used as a proxy**.

This design ensures:

- Stability of execution
- Demonstration of adversarial vulnerability concept

The goal was to **simulate adversarial conditions rather than optimize attack strength**.

Conclusion

The system prioritizes **robust integration, computational feasibility, and architectural completeness** over individual module perfection. Each design choice reflects a trade-off driven by dataset limitations, environment constraints, and assignment objectives.